

FUNDAÇÃO ESCOLA DE COMÉRCIO ÁLVARES PENTEADO
FECAP
CENTRO UNIVERSITÁRIO ÁLVARES PENTEADO
ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

Cauã Santos Nascimento
Pedro Henrique Santos da Silva
Paulo Miguel Miranda Marcelli
Gabriel Vaz Ferreira Neves

PROJETO INTEGRADO
ALGORITMOS E LÓGICA DE PROGRAMAÇÃO

PROFESSOR(A): LUCY MARI TABUTI

São Paulo
2026

Sumário

1. Introdução	3
2. Estrutura de Repetição.....	4
2.1 Entrada e saída	4
3. Estrutura Sequencial	5
3.1 Entrada e saída	5
4. Estrutura de Decisão.....	7
4.1 Entrada e saída	7

1. Introdução

Este documento apresenta a implementação das três estruturas fundamentais da programação, sequencial, repetição e decisão aplicadas ao projeto SchoolPlayManager, desenvolvido em C# com Windows Forms. O sistema realiza simulação de acesso por IPv4, à plataforma criada, validando endereços e registrando logs de acesso. Cada seção demonstra como essas estruturas funcionam no sistema, com exemplos visuais e explicações sobre o seu funcionamento.

2. Estrutura de Repetição

A estrutura de repetição permite que um bloco de código seja executado múltiplas vezes, de forma controlada, enquanto uma condição for verdadeira. No projeto SchoolPlayManager, ela é utilizada para percorrer a lista de IPs autorizados e verificar se o IP digitado pelo usuário está presente nessa lista.

2.1 Entrada e saída

Entrada: o IP digitado pelo usuário, passado como parâmetro (ipTentativa) para o método ValidarAcesso().

Saída: um valor booleano — true se o IP for encontrado na lista, ou false se o laço terminar sem encontrá-lo.

Trecho do código — Classe IPValidator (camada Business):

```
//Ips para simulação do acesso
private List<string> _ipsAutorizados = new List<string> { "192.168.1.50", "172.16.0.10" };

public bool ValidarAcesso(string ipTentativa)
{
    for (int i = 0; i < _ipsAutorizados.Count; i++)
    {
        if (_ipsAutorizados[i] == ipTentativa)
        {
            return true;
        }
    }
}
```

O laço for inicializa a variável i em 0 e itera enquanto i for menor que o total de IPs na lista (_ipsAutorizados.Count), incrementando i a cada volta. Em cada iteração, o código compara o IP na posição i com o IP digitado pelo usuário. Se forem iguais, o método retorna true imediatamente, encerrando o laço antes do fim. Caso o laço complete todas as iterações sem encontrar o IP, o método retorna false, indicando que o acesso não está autorizado.

Os IPs configurados para simulação são: 192.168.1.50 e 172.16.0.10. Qualquer outro endereço resultará em acesso bloqueado.

3. Estrutura Sequencial

A estrutura sequencial é a mais básica da programação: as instruções são executadas uma após a outra, na ordem em que aparecem no código, sem desvios ou repetições. Cada linha depende da execução correta da linha anterior para funcionar corretamente.

3.1 Entrada e saída

Entrada: nenhuma entrada do usuário neste trecho — o construtor é executado automaticamente quando o formulário é aberto pelo sistema.

Saída: o formulário é exibido na tela já com os componentes visuais prontos e com a validação inicial executada, mostrando a mensagem "Formato de IP Inválido!" pois o campo está vazio.

Trecho do código — Construtor da classe AccessSimulationForm:

```
public partial class AccessSimulationForm : Form
{
    public AccessSimulationForm()
    {
        InitializeComponent();

        // Optional: trigger the verification once the form components are initialized
        // (original code called the handler with null parameters at file scope)
        btnVerificarAcesso_Click(null, null);

        // Centraliza os controles horizontalmente em relação ao botão
        int centroX = btnVerificarAcesso.Left + btnVerificarAcesso.Width / 2;
        lblStatus.Left = centroX - lblStatus.Width / 2;

        txtIpOrigem.Width = btnVerificarAcesso.Width;
        txtIpOrigem.Left = centroX - txtIpOrigem.Width / 2;

        // Label com largura maior para o texto caber
        lblStatus.AutoSize = true;
        lblStatus.Width = 250; // ajuste esse valor se precisar
        lblStatus.Left = centroX - lblStatus.Width / 2;
        lblStatus.TextAlign = ContentAlignment.MiddleCenter;

        teste.Width = btnVerificarAcesso.Width;
        teste.Left = centroX - teste.Width / 2;

        btnVerificarAcesso_Click(null, null);
    }
}
```

O construtor executa as instruções na seguinte ordem sequencial: primeiro, `InitializeComponent()` é chamado, criando e posicionando todos os elementos visuais do formulário (botões, caixas de texto, labels e lista). Somente após essa inicialização, `btnVerificarAcesso_Click(null, null)` é chamado para executar a validação inicial com o campo vazio.

Em seguida, as instruções de alinhamento são executadas uma a uma: o centro horizontal é calculado com base na posição do botão, e cada controle visual (campo de texto, label e lista) tem sua largura e posição ajustadas sequencialmente. Inverter qualquer uma dessas ordens causaria erros, pois não seria possível centralizar um elemento antes de conhecer a posição do botão de referência.

4. Estrutura de Decisão

A estrutura de decisão permite que o programa execute diferentes blocos de código dependendo de uma condição. No projeto SchoolPlayManager, ela é utilizada para determinar se o acesso do IP deve ser autorizado ou bloqueado e, com base nisso, atualizar a interface visual com a mensagem e a cor correspondentes.

4.1 Entrada e saída

Entrada: a variável booleana `isAutorizado`, que contém o resultado retornado pelo método `ValidarAcesso()` da camada Business.

Saída: atualização da label `lblStatus` na tela — texto "Acesso Autorizado" em verde caso o IP esteja na lista, ou "Acesso Bloqueado" em vermelho caso contrário.

Trecho do código — Método `btnVerificarAcesso_Click` (camada de apresentação):

```
// 3. ATUALIZAÇÃO DA TELA
if (isAutorizado)
{
    lblStatus.Text = "Acesso Autorizado";
    lblStatus.ForeColor = Color.Green;
}
else
{
    lblStatus.Text = "Acesso Bloqueado";
    lblStatus.ForeColor = Color.Red;
}
```

O bloco `if` verifica o valor de `isAutorizado`. Se verdadeiro (`true`), a label recebe o texto "Acesso Autorizado" e a cor verde (`Color.Green`), fornecendo feedback visual positivo ao usuário. Se falso (`false`), o bloco `else` é executado, definindo o texto como "Acesso Bloqueado" e a cor como vermelho (`Color.Red`).

Apenas um dos dois blocos é executado a cada chamada — nunca ambos ao mesmo tempo. Essa estrutura garante que a interface sempre reflita o resultado correto da validação realizada pela camada de negócios.